

I. Objective

Develop a real-time data compression and decompression system for satellite communication using two FPGAs. Implement the RLE compression algorithm in VHDL on the transmitting FPGA and establish a high-speed UART serial interface to transfer the compressed data to the receiving FPGA, where it is decompressed in real time. Optimize the system for low latency and efficient bandwidth utilization.

II. Procedure

A) Image Inclusion

- Image needs to be converted into a .coe file. A .coe file represents the hexadecimal/binary representation of each pixel in the image.
- A Matlab code is implemented to generate the binary data of the pixel.
- To match the compatibility of the FPGA, the image is converted into an 8 bit .coe file.
- The image is stored in the FPGA in the Block RAM (BRAM) with width 12 bits. For the image used, it needed 17 bits for addressing. The IP Catalog was used to include the image coe file into the project.

B) Compression and Decompression

- **Run-Length Encoding (RLE)** is a fundamental lossless data compression algorithm that is particularly effective for images with large regions of uniform color or intensity. It operates by identifying sequences of repeating values and representing them as a single value followed by a count, thereby reducing the overall data size.

For example: AA,AA,FE,AA=> AA x 2,FE x 1,AA x 1

- In digital images, pixel values often repeat, especially in areas with uniform color. RLE exploits this redundancy by replacing consecutive identical pixel values with a pair:

→ Run Value: the repeated pixel value/data value.

→ Run Length: the number of consecutive repetitions.

- **Implementation in Digital Systems**

In a digital system design context, RLE can be implemented using a simple sequential logic that performs the following steps:

1. **Input Pixel Stream:** Accepts pixel data serially.
2. **Run Detection:** Compares the current pixel with the previous one.
3. **Counter:** Increments a counter for as long as the pixel value remains unchanged.

4. **Output Pair:** Once the value changes, it outputs the current value and run length, then resets the counter.

- Advantages in Image Compression

- Simplicity: Easy to implement in VHDL/Verilog for FPGA-based systems.
- Efficiency: Performs well on binary and grayscale images with many repeated values.
- Low Resource Usage: Minimal logic and memory requirements make it ideal for real-time embedded systems.

- Limitations

- Ineffective on Noisy or High-Entropy Images: RLE may increase the size of the data if the image has high pixel variability.
- Fixed-Length Runs: For very long runs, fixed-width counters may limit maximum run length unless properly managed.

- Compression Ratio

Original Image Size: 153600 bits

Number of RLE pairs: 1048

Compressed size = Number of RLE pairs * 18 bits/pair = 18864 bits

Compression Ratio = 153600 bits / 18864 bits = 8.14

C) UART Communication Between Two FPGAs

To enable communication between the compression and decompression modules implemented on two separate FPGA boards, a Universal Asynchronous Receiver/Transmitter (UART) interface was used. This provided a straightforward and effective means of serial data transmission, facilitating synchronized data exchange during the image compression and decompression process.

- Implementation Steps

The UART communication was incrementally developed, beginning with basic functionality:

- Initial Testing: Switch toggles on the transmitter FPGA were used to send single-byte data to the receiver FPGA.
- Control Logic: A dedicated *send button* was introduced to initiate transmission, while a *reset button* ensured proper reinitialization of the UART states and buffers.

- Technical Challenges and Solutions

Throughout development, several hardware and configuration-related issues were encountered:

→ ADC Port Conflict: Initially, the UART line was connected to an Analog-to-Digital Converter (ADC) port, which was not configured for digital I/O. This was corrected by explicitly defining the pin as a standard I/O in the XDC constraints file.

→ Pin Mismatch and Constraints: Careful attention was required to match the TX and RX lines between the boards and ensure proper constraint definitions for both sending and receiving ends.

→ Debugging via 7-Segment Display: To verify correct data transmission, the received compressed values were displayed on a 7-segment display. This helped confirm that both the data and its corresponding length were correctly received.

→ Using a switch to send the data: Using one switch, one line connection, a LED and two buttons. The switch will allow the user to enter either a 0/1 the button will give the order to send the bit through the channel from an FPGA to the other, the LED is a signal that states that the user sent a byte through the channel and finally the second button for resetting the binary in case that the data is wrong so its send an array of zero and resets.

- Final Design

The finalized UART communication system featured the following:

→ Data Structure: The compressed output was split into two components:

- Data byte (literal or match information)
- Length byte (for matched sequences)

Each component was transmitted via separate UART ports for synchronization and modularity.

→ Hardware Considerations:

- A common GND connection was essential between the two FPGA boards to establish a shared reference for signal integrity.
- The VCC lines were deliberately not connected to prevent potential current backflow and overheating, ensuring safe operation.

D) VGA : Video Graphics Array

It's a port that isn't like the Pmod ports. Its function is to display an image either grayscale or red green and blue (aka RGB). The VGA is so basic and simple in terms of pixel and quality. It provides only the maximum RGB as a 12 bit which means each color will be represented with only 4 bits. The 4 bits explain how bright is the color that you try to display. Recently, we have discussed that the VGA has 12 ports that flow signals but it is incorrect, it has 14 ports to display the pixel.

What are the 2 ports left?

There is the *h_sync* and the *v_sync*.

What are the *h_sync* and the *v_sync*? They are the ones that tell if it's possible to display in the range of the resolution in the X and Y.

After defining what is the VGA? what does it constitute and its role? We can start to dive into our project that is to display the image compressed/decompressed.

At first, we decided to understand how to display a solid color and understand the logic beyond displaying a solid color/ grayscale / colored image.

Of course we can't display an image on the VGA directly without passing on the vga timing graph that differs from one screen to the other.

The VGA timing consists of two parts:

1- the Fame: the vertical synchronization which is the combinations of:

Front/Back Porph, Vertical Display Area.

2- Scanline : the horizontal synchronization which is the combinations of :

Frot/Back Porph, Horizontaal Display Area.

Let's take for example the most basic vga timing:

Vertical:	vs	Horizontal:
Front Porph:10		16
Back Porph: 33		48
Display Area: 480		640

But in general, you have to display an image on a screen resolution equally or less than the PC/Screen that you have.

With the situation that we have to deal with, we have to display on a screen with a 1920x1080 , we had to use a smaller resolution so that it could be displayed.

E) Predefined Vivado codes :

These predefined Vivado tools are located in the Project Manager=> IP Catalog

Some tools are needed such as :

- importing a coe.file:

Importing an image converted to Binary/Hexadecimal isn't that simple. It also requires precise pixel width , pixel depth and indications about the file(it must fit the number of bits for each pixel and the resolution that it will be displayed).

- building an empty bram to store x bits with y resolutions:

Sometimes during displaying in a VGA, people are encountering issues : might they didn't store the data somewhere, might it had displayed once and then it disappeared. So storing it will allow you to do multiple tasks with the same data.

- modifying the default clock of the FPGA(default : 100 MHz):

Some codes will accept to run with the default frequencies, others will require a slower/faster clock to synchronize the data transfer or even limit the time of processing. If i m changing the period(it's related to the clock) , the data is sent faster but there is a limit, because the fpga will not support this frequency and clock.

We can set the limits of the FPGA:

The FPGA can support any frequency between 100 to 150 MHz after that the device will get damaged, and the clock should and must not be below 10 ns . The standard clock for the FPGA is 20 ns the minimum timing is 10/9 ns.

- Image memory type:

We know that memory is divided into two categories: RAM(Random Access Memory) and ROM(Read Only Memory). This is the standard one that you encounter in your real life, but in the Vivado, it's special. You have to specify the category the sub-category, e.g. Single/Simple Dual/True Dual Port RAM and Single/Dual Port ROM:

Let's go through an example: In the project of displaying data, it was required from the student to store the data in a Simple Dual Port RAM. This category of memory has two ports :the first one optional (Port A) that specifies if the data will be Read/Write and the second one(Port B) is a non-optional "write " .

III. Project Codes and Simulations:

A. FPGA I:

1. VGA:

a. Code implementation:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity VGA is
  Port (
    clk_25MHz : in  STD_LOGIC;
    reset      : in  STD_LOGIC;
    hsync      : out STD_LOGIC;
    vsync      : out STD_LOGIC;
    red        : out STD_LOGIC_VECTOR(2 downto 0);
    green      : out STD_LOGIC_VECTOR(2 downto 0);
    blue       : out STD_LOGIC_VECTOR(1 downto 0);
    uart_tx    : out STD_LOGIC;  -- UART TX Output
    tx_out     : out STD_LOGIC_VECTOR(17 downto 0) -- Optional debug
  output
  );
end VGA;

architecture Behavioral of VGA is

  component image_disp
    port(
      clka  : IN STD_LOGIC;
      addra : IN STD_LOGIC_VECTOR(14 DOWNTO 0);
      douta : OUT STD_LOGIC_VECTOR(7 downto 0)
    );
  end component;

  component rle_encoder
    port (
      clk      : in  STD_LOGIC;
      rst      : in  STD_LOGIC;
      data_in   : in  STD_LOGIC_VECTOR(7 downto 0);
      valid_in  : in  STD_LOGIC;
      code_out  : out STD_LOGIC_VECTOR(17 downto 0);
      valid_out : out STD_LOGIC
    );
  end component;

  -- VGA timing constants (1600x900)
```

```

constant H_VISIBLE : integer := 1600;
constant H_TOTAL   : integer := 1760;
constant V_VISIBLE : integer := 900;
constant V_TOTAL   : integer := 932;

-- VGA signals
signal h_count, v_count : integer range 0 to 2500 := 0;
signal pixel_addr       : std_logic_vector(14 downto 0);
signal pixel_data       : std_logic_vector(7 downto 0);

-- RLE signals
signal rle_code         : std_logic_vector(17 downto 0);
signal rle_valid        : std_logic;
signal rle_read_addr    : std_logic_vector(14 downto 0);
signal rle_write_addr : std_logic_vector(14 downto 0) := (others => '0');

component uart_txs
  Port (
    clk      : in  std_logic;
    rst      : in  std_logic;
    tx_start  : in  std_logic;
    tx_data   : in  std_logic_vector(7 downto 0);
    tx_out    : out std_logic;
    busy      : out std_logic
  );
end component;

-- ... [no changes to image_disp or rle_encoder components]

-- UART signals
signal uart_tx_data      : std_logic_vector(7 downto 0);
signal uart_tx_start     : std_logic := '0';
signal uart_tx_out       : std_logic;
signal uart_state        : integer range 0 to 3 := 0;
signal uart_buffer       : std_logic_vector(17 downto 0);
signal uart_busy         : std_logic;

begin

  uart_tx <= uart_tx_out;
  tx_out <= uart_buffer;

  -- Instantiate BRAM
  image_bram_inst: image_disp
    port map (
      clka => clk_25MHz,

```

```

        addra => pixel_addr,
        douta => pixel_data
    );

-- Instantiate RLE encoder
rle_inst: rle_encoder
    port map (
        clk        => clk_25MHz,
        rst        => reset,
        data_in     => pixel_data,
        valid_in    => '1',
        code_out    => rle_code,
        valid_out   => rle_valid
    );

-- Instantiate UART transmitter
uart_inst: uart_txs
    port map (
        clk        => clk_25MHz,
        rst        => reset,
        tx_start   => uart_tx_start,
        tx_data    => uart_tx_data,
        tx_out     => uart_tx_out,
        busy       => uart_busy
    );

-- VGA pixel address logic
process(h_count, v_count)
    variable img_x, img_y : integer;
begin
    if h_count < H_VISIBLE and v_count < V_VISIBLE then
        img_x := h_count / 5;
        img_y := v_count / 4;
        if img_x < 160 and img_y < 120 then
            pixel_addr <= std_logic_vector(to_unsigned(img_y * 160 +
img_x, 15));
        else
            pixel_addr <= (others => '0');
        end if;
    else
        pixel_addr <= (others => '0');
    end if;
end process;

-- VGA output (basic grayscale)
process(rle_code)

```



```

begin
    red    <= rle_code(17 downto 15);
    green  <= rle_code(14 downto 12);
    blue   <= rle_code(11 downto 10);
end process;

-- VGA counters
process(clk_25MHz)
begin
    if rising_edge(clk_25MHz) then
        if reset = '1' then
            h_count <= 0;
            v_count <= 0;
        else
            if h_count = H_TOTAL - 1 then
                h_count <= 0;
                if v_count = V_TOTAL - 1 then
                    v_count <= 0;
                else
                    v_count <= v_count + 1;
                end if;
            else
                h_count <= h_count + 1;
            end if;
        end if;
    end if;
end process;

-- Sync signals
hsync <= '0' when (h_count >= H_VISIBLE + 48 and h_count < H_VISIBLE +
80) else '1';
vsync <= '0' when (v_count >= V_VISIBLE + 4 and v_count < V_VISIBLE + 9)
else '1';

-- UART FSM for sending RLE data in 3 bytes
-- UART FSM for sending RLE data in 3 bytes
process(clk_25MHz)
begin
    if rising_edge(clk_25MHz) then
        if reset = '1' then
            uart_state <= 0;
            uart_tx_start <= '0';
            uart_tx_data <= (others => '0');
            uart_buffer <= (others => '0');
        else
            case uart_state is

```

```

when 0 =>
    uart_tx_start <= '0';
    if rle_valid = '1' and uart_busy = '0' then
        uart_buffer <= rle_code;
        uart_tx_data <= rle_code(17 downto 10);
        uart_tx_start <= '1';
        uart_state <= 1;
    end if;

when 1 =>
    uart_tx_start <= '0';
    if uart_busy = '0' then
        uart_tx_data <= uart_buffer(9 downto 2);
        uart_tx_start <= '1';
        uart_state <= 2;
    end if;

when 2 =>
    uart_tx_start <= '0';
    if uart_busy = '0' then
        uart_tx_data <= uart_buffer(1 downto 0) &
"000000";

        uart_tx_start <= '1';
        uart_state <= 3;
    end if;

when 3 =>
    uart_tx_start <= '0';
    if uart_busy = '0' then
        uart_state <= 0;
    end if;

when others =>
    uart_state <= 0;
end case;
end if;
end if;
end process;

end Behavioral;

```

b. TestBench:

We can't testbench it since we are not feeding data , so we will with the fact that the data will be displayed



2. RLE_Compression:

a. Code implementation:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity rle_encoder is
    port (
        clk          : in  std_logic;
        rst          : in  std_logic;
        data_in       : in  std_logic_vector(7 downto 0); -- 8-bit input
        valid_in      : in  std_logic;
        code_out      : out std_logic_vector(17 downto 0); -- 8-bit data +
10-bit run count
        valid_out     : out std_logic
    );
end entity rle_encoder;

architecture rtl of rle_encoder is
    signal current_byte : std_logic_vector(7 downto 0) := (others => '0');
    signal run_count    : unsigned(9 downto 0) := (others => '0');
    signal busy         : std_logic := '0';
    signal prev_valid   : std_logic := '0';
begin
    process(clk)
    begin
```

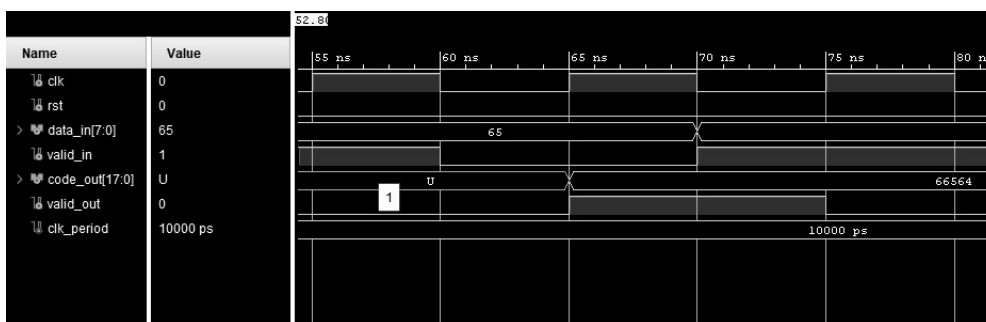
```

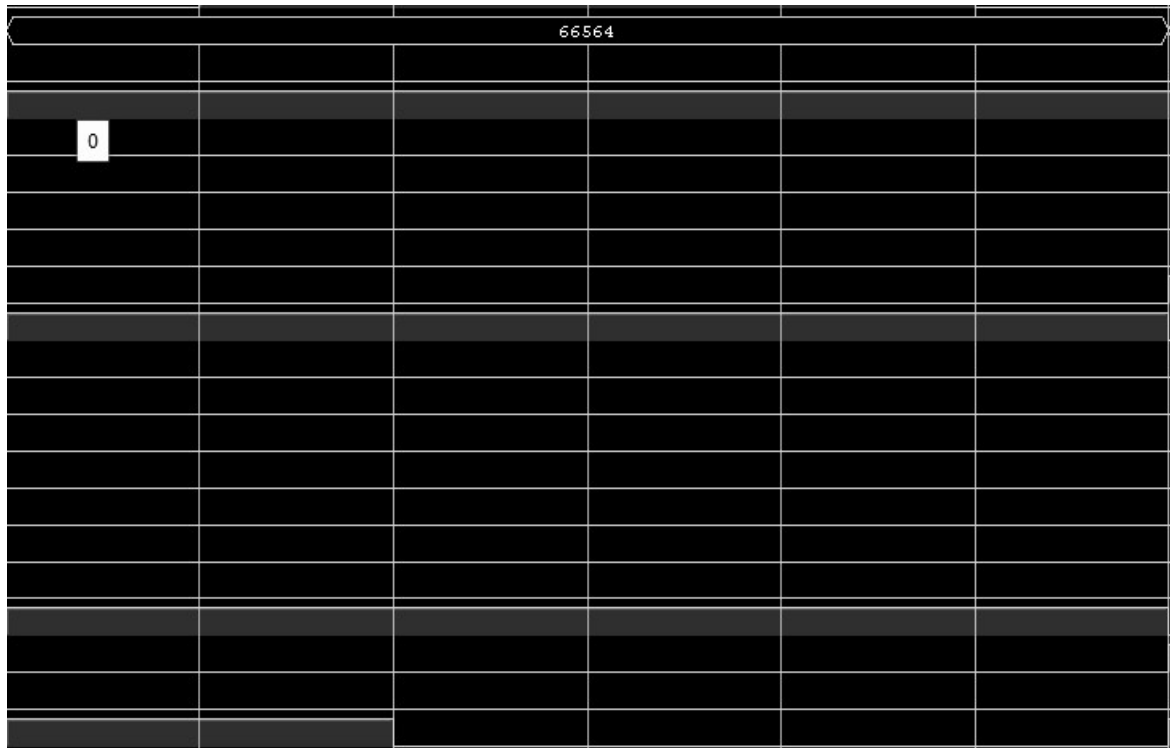
if rising_edge(clk) then
  if rst = '1' then
    current_byte <= (others => '0');
    run_count    <= (others => '0');
    busy        <= '0';
    valid_out    <= '0';
    prev_valid   <= '0';
  else
    prev_valid <= valid_in;
    valid_out  <= '0';

    if valid_in = '1' then
      if busy = '0' then
        busy        <= '1';
        current_byte <= data_in;
        run_count    <= to_unsigned(1, run_count'length);
      elsif data_in = current_byte and run_count <
to_unsigned(1023, run_count'length) then
        run_count <= run_count + 1;
      else
        code_out    <= current_byte &
std_logic_vector(run_count);
        valid_out    <= '1';
        current_byte <= data_in;
        run_count    <= to_unsigned(1, run_count'length);
      end if;
    elsif prev_valid = '1' and busy = '1' then
      code_out    <= current_byte & std_logic_vector(run_count);
      valid_out    <= '1';
      busy        <= '0';
    end if;
  end if;
end if;
end process;
end architecture rtl;

```

b. Testbench:





Since the RLE takes the data(8 bits) and outputs the data (18 bits). In the above waveform of the test bench, we can notice that when the data_in is 65, it will output a binary value with the upperbits, which will be the 8 bits(17 down to 10) and the lower bits, which will be the first 10(9 down to 0).

Lets derive that;

Input : 65(Hexadecimal)

Output: 66564(hexadecimal)

The valid_in flag will input the rle component and read the data entered. It will calculate how many of this input had been redundant sequentially. It found out the output

code_out:01000001/0000001000

What we extracted from this waveform is that the input fed into the rle_compression is 65:01000001 and the running length is 8:00000001000.

English wise:”The hexadecimal number 65 has been found 8 times in a row,including the first one”.

At the beginning of the output the data sets the valid_out flag as true and then makes it false, because it will allow the data to be analysed and go through the output ports.

2. Uart: Transmission part:

a. Code implementation:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity uart_txs is
    Port (
        clk      : in  std_logic;
        rst      : in  std_logic;
        tx_start  : in  std_logic;
        tx_data   : in  std_logic_vector(7 downto 0);
        tx_data1  : out std_logic_vector(17 downto 0); -- passthrough/debug
        tx_out    : out std_logic;
        busy      : out std_logic                    -- HIGH when
        transmitting
    );
end uart_txs;
```

architecture Behavioral of uart_txs is

```
    constant CLKS_PER_BIT : integer := 868; -- for 115200 baud @ 50 MHz

    type state_type is (IDLE, START_BIT, DATA_BITS, STOP_BIT, CLEANUP);
    signal state       : state_type := IDLE;

    signal clk_count    : integer range 0 to CLKS_PER_BIT := 0;
    signal bit_index    : integer range 0 to 7 := 0;
    signal tx_byte      : std_logic_vector(7 downto 0) := (others => '0');
    signal tx_reg       : std_logic := '1';
    signal sending      : std_logic := '0';

begin

    tx_out    <= tx_reg;
    tx_data1  <= (others => '0'); -- Optional output; you can drive with
tx_byte or debug
    busy      <= sending;

    process(clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                state    <= IDLE;
                clk_count <= 0;
            end if
        end if
    end process;
```

```

    bit_index <= 0;
    tx_reg    <= '1';
    sending   <= '0';

else
    case state is

        when IDLE =>
            tx_reg <= '1';
            sending <= '0';
            if tx_start = '1' then
                tx_byte <= tx_data;
                sending <= '1';
                state <= START_BIT;
            end if;

        when START_BIT =>
            tx_reg <= '0'; -- Start bit
            if clk_count < CLKS_PER_BIT - 1 then
                clk_count <= clk_count + 1;
            else
                clk_count <= 0;
                state <= DATA_BITS;
                bit_index <= 0;
            end if;

        when DATA_BITS =>
            tx_reg <= tx_byte(bit_index);
            if clk_count < CLKS_PER_BIT - 1 then
                clk_count <= clk_count + 1;
            else
                clk_count <= 0;
                if bit_index < 7 then
                    bit_index <= bit_index + 1;
                else
                    bit_index <= 0;
                    state <= STOP_BIT;
                end if;
            end if;

        when STOP_BIT =>
            tx_reg <= '1';
            if clk_count < CLKS_PER_BIT - 1 then
                clk_count <= clk_count + 1;
            else
                clk_count <= 0;
            end if;
    end case;
end if;

```


B. FPGA II:

1. VGA:

a. Code Implementation:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity VGA_2 is
    Port (
        clk_25MHz      : in  STD_LOGIC;
        reset          : in  STD_LOGIC;
        compressed_data_in : in STD_LOGIC_VECTOR(17 downto 0);
        compressed_valid : in STD_LOGIC;
        hsync           : out STD_LOGIC;
        vsync           : out STD_LOGIC;
        red              : out STD_LOGIC_VECTOR(2 downto 0);
        green            : out STD_LOGIC_VECTOR(2 downto 0);
        blue             : out STD_LOGIC_VECTOR(1 downto 0)
    );
end VGA_2;

architecture Behavioral of VGA_2 is

    -- VGA Timing Constants
    constant H_VISIBLE   : integer := 1600;
    constant H_FRONT     : integer := 48;
    constant H_SYNC      : integer := 32;
    constant H_BACK      : integer := 80;
    constant H_TOTAL     : integer := H_VISIBLE + H_FRONT + H_SYNC +
H_BACK;

    constant V_VISIBLE   : integer := 900;
    constant V_FRONT     : integer := 4;
    constant V_SYNC      : integer := 5;
    constant V_BACK      : integer := 23;
    constant V_TOTAL     : integer := V_VISIBLE + V_FRONT + V_SYNC +
```

V_BACK;

-- Component Declarations

component rle_reading_18bits

PORT (

clka : IN STD_LOGIC;

wea: in std_logic;

addra : IN STD_LOGIC_VECTOR(14 DOWNT0 0);

dina : IN STD_LOGIC_VECTOR(17 DOWNT0 0);

clkb : IN STD_LOGIC;

addrb : IN STD_LOGIC_VECTOR(14 DOWNT0 0);

doutb : OUT STD_LOGIC_VECTOR(17 DOWNT0 0)

);

end component;

component rle_reading_8bits

PORT (

clka : IN STD_LOGIC;

wea :in std_logic;

addra : IN STD_LOGIC_VECTOR(14 DOWNT0 0);

dina : IN STD_LOGIC_VECTOR(7 DOWNT0 0);

clkb : IN STD_LOGIC;

addrb : IN STD_LOGIC_VECTOR(14 DOWNT0 0);

doutb : OUT STD_LOGIC_VECTOR(7 DOWNT0 0)

);

end component;

component RLE_decompressor

port (

clk : in std_logic;

rst : in std_logic;

start : in std_logic;

code_in : in std_logic_vector(17 downto 0);

valid_in : in std_logic;

data_out : out std_logic_vector(767 downto 0); -- match real

decompressor

done : out std_logic -- match real

decompressor

);

end component;

-- Signals

```

    signal h_count, v_count : integer range 0 to 2500 := 0;
    signal pixel_addr       : std_logic_vector(14 downto 0);
    signal pixel_data       : std_logic_vector(7 downto 0);

    signal rle18_addr       : std_logic_vector(14 downto 0) := (others
=> '0');
    signal rle18_dout       : std_logic_vector(17 downto 0);
    signal rle8_addr        : std_logic_vector(14 downto 0) := (others
=> '0');
    signal rle8_din         : std_logic_vector(7 downto 0);
    signal rle8_we          : std_logic := '0';

    signal decompress_start : std_logic := '0';
    signal decompress_valid : std_logic := '0';
    signal decompress_output: std_logic_vector(7 downto 0);
    signal decompress_done  : std_logic;
    signal decompress_state : integer range 0 to 2 := 0;

    signal decompressed_buffer : std_logic_vector(767 downto 0) :=
(others => '0');
    signal output_counter : integer range 0 to 95 := 0; -- 768/8 = 96
bytes

    -- Address to store compressed data
    signal rle18_write_addr : std_logic_vector(14 downto 0) := (others
=> '0');
    signal rle18_wea       : std_logic := '0';

```

begin

```

RLE18_RAM_inst : rle_reading_18bits
    port map (
        clk_a => clk_25MHz,
        wea => '0',

        addra => rle18_write_addr,
        dina  => compressed_data_in,
        clk_b => clk_25MHz,
        addrb => rle18_addr,
        doutb => rle18_dout
    );

```

```

RLE8_RAM_inst : rle_reading_8bits
  port map (
    clka => clk_25MHz,
    wea => '0',
    addra => rle8_addr,
    dina => rle8_din,
    clkb => clk_25MHz,
    addrb => pixel_addr,
    doutb => pixel_data
  );

-- Instantiate Decompressor
-- Instantiate Decompressor
decompressor_inst : RLE-Decompressor
  port map (
    clk      => clk_25MHz,
    rst      => reset,
    start    => decompress_start,
    code_in  => rle18_dout,
    valid_in => decompress_valid,
    data_out => decompressed_buffer, -- Store full 768 bits
    done     => decompress_done
  );

-- Decompression Logic
process(clk_25MHz)
begin
  if rising_edge(clk_25MHz) then
    decompress_start <= '0';
    rle8_we          <= '0';

    case decompress_state is
      when 0 => -- Start decompression
        decompress_valid <= '1';
        decompress_start <= '1';
        decompress_state <= 1;

      when 1 => -- Wait for decompressor output
        if decompress_done = '1' then
          rle8_we    <= '1';
          rle8_din   <= decompress_output;
          rle8_addr <=
std_logic_vector(unsigned(rle8_addr) + 1);

```

```

        decompress_state <= 2;
    end if;

    when 2 => -- Move to next RLE word
        decompress_valid <= '0';
        rle18_addr <= std_logic_vector(unsigned(rle18_addr)
+ 1);

        decompress_state <= 0;
    end case;
end if;
end process;

-- Write compressed data to RLE18 RAM
process(clk_25MHz)
begin
    if rising_edge(clk_25MHz) then
        if compressed_valid = '1' then
            rle18_wea <= '1';
            rle18_write_addr <=
std_logic_vector(unsigned(rle18_write_addr) + 1);
        else
            rle18_wea <= '0';
        end if;
    end if;
end process;

-- VGA Address Mapping for 1600x900 resolution
process(h_count, v_count)
    variable img_x, img_y : integer;
begin
    if h_count < H_VISIBLE and v_count < V_VISIBLE then
        img_x := h_count / 4;
        img_y := v_count / 3;
        if img_x < 160 and img_y < 120 then
            pixel_addr <= std_logic_vector(to_unsigned(img_y *
160 + img_x, 15));
        else
            pixel_addr <= (others => '0');
        end if;
    else
        pixel_addr <= (others => '0');
    end if;
end process;

```

```

end process;

-- Map grayscale data to RGB
process(pixel_data)
begin
    red    <= pixel_data(7 downto 5);
    green  <= pixel_data(4 downto 2);
    blue   <= pixel_data(1 downto 0);
end process;

-- VGA Timing Logic
process(clk_25MHz, reset)
begin
    if rising_edge(clk_25MHz) then
        if reset = '1' then
            h_count <= 0;
            v_count <= 0;
        else
            if h_count = H_TOTAL - 1 then
                h_count <= 0;
                if v_count = V_TOTAL - 1 then
                    v_count <= 0;
                else
                    v_count <= v_count + 1;
                end if;
            else
                h_count <= h_count + 1;
            end if;
        end if;
    end if;
end process;

-- Sync Signal Generation
hsync <= '0' when (h_count >= H_VISIBLE + H_FRONT and h_count
< H_VISIBLE + H_FRONT + H_SYNC) else '1';
vsync <= '0' when (v_count >= V_VISIBLE + V_FRONT and v_count
< V_VISIBLE + V_FRONT + V_SYNC) else '1';

end Behavioral;

```